

Lab 4 – Working with JSON in Python

Part 0: Setup

1. Open Your Project:

- Ensure you are in your VS Code window, connected to the "Dev Container: Python 3" for your `Day2_Python_Labs` project.

2. Create a New Script:

- In the VS Code Explorer, create a new Python file named `json_lab.py`.
- Add the `json` module import at the top. This module is part of Python's standard library, so no `pip install` is needed.

```
import json
```

Part 1: Parsing a JSON String into a Python Object

1. Define a JSON String:

- Imagine you received the following data from an IoT device configuration API. It's currently a multi-line string in your Python script.
- Copy this into `json_lab.py`:

```
device_config_json_string = """
{
  "deviceId": "sensor-hub-01",
  "location": {
    "building": "A",
    "floor": 3,
    "room": "302"
  },
  "isActive": true,
  "sensors": [
    {
      "type": "temperature",
      "unit": "celsius",
      "port": 1,
      "lastReading": null
    },
    {
      "type": "humidity",
      "unit": "%",
      "port": 2,
      "lastReading": 65.5
    }
  ]
}
```

```
}  
]  
}  
"""
```

2. Load the JSON String:

- Use the `json.loads()` function (load string) to parse the string into a Python object (which will be a dictionary).

```
# Parse the JSON string into a Python dictionary  
device_data = json.loads(device_config_json_string)
```

3. Explore the Python Object:

- Now that you have a Python dictionary, you can work with it like any other.
- Add these lines to inspect the data:

```
print("--- Exploring Parsed JSON Data ---")  
print(f"The data type is: {type(device_data)}")  
  
# Access top-level keys  
device_id = device_data['deviceId']  
print(f"Device ID: {device_id}")  
  
# Access nested object data  
room_number = device_data['location']['room']  
print(f"Room Number: {room_number}")  
  
# Access data from a list of objects  
first_sensor_type = device_data['sensors'][0]['type']  
print(f"First sensor type: {first_sensor_type}")  
  
# Loop through the sensors  
print("\nAll sensors:")  
for sensor in device_data['sensors']:  
    print(f"    - Type: {sensor['type']}, Port: {sensor['port']}")
```

4. Run Your Script:

- Execute `python json_lab.py` in the integrated terminal and verify the output.

Part 2: Modifying the Python Object and Serializing Back to JSON

1. Update the Data:

- Now, let's modify the `device_data` dictionary in memory.

```

print("\n--- Modifying Python Object ---")

# Update a value
device_data['isActive'] = False
print("Set isActive to False.")

# Update a nested value
device_data['location']['room'] = "305-B"
print("Moved device to room 305-B.")

# Update a value in the sensor list
device_data['sensors'][0]['lastReading'] = 22.7
print("Updated temperature sensor's last reading.")

# Add a new key-value pair
device_data['firmwareVersion'] = "v1.2.3"
print("Added firmware version.")

```

2. Serialize Back to a JSON String:

- Use the `json.dumps()` function (dump string) to convert your modified Python dictionary back into a JSON formatted string.

```

print("\n--- Serializing back to JSON String ---")

# Convert the dictionary back to a JSON string with nice formatting
updated_json_string = json.dumps(device_data, indent=2)

print("Updated JSON Configuration:")
print(updated_json_string)

```

- Explanation:
 - `json.dumps()` : The function to serialize a Python object to a JSON string.
 - `indent=2` : This is a crucial argument for readability. It tells `dumps` to format the output string with an indentation of 2 spaces, making it look like the original.

3. (Bonus) Save to a File:

- Let's save this new configuration to a file.

```

# Save the updated JSON to a file
file_path = "updated_config.json"
with open(file_path, "w") as json_file:
    json_file.write(updated_json_string)

```

```
print(f"\nSuccessfully saved updated configuration to {file_path}")
```

4. Run and Verify:

- Run `python json_lab.py` again.
- Check the output in your terminal.
- Look in the VS Code Explorer. You should see a new file, `updated_config.json`. Open it to see your modified, nicely formatted JSON.